

A Layered Framework for Thinking About AI & LLM Security

Six defensible layers, from the network up to how the model itself behaves — each one grounded in a named industry framework, and checked against what DeepTutor actually does today.

Subject – DeepTutor, a multi-user AI tutoring platform

Audience – a security-minded reviewer

Jurisdiction – Uganda (deployment), cross-referenced against South Africa

THE SHORT VERSION

Security for an LLM application isn't one thing — it's the same layered thinking you'd apply to any system (network, infrastructure, data), plus **two entirely new layers** that didn't exist before language models: how the model can be manipulated through language itself, and how much autonomy you hand an AI agent to act on its own. The first four layers below are largely solved problems with known playbooks. The last two are genuinely new, actively being standardized right now (the agentic framework below is barely two months old), and is where "we've never talked about this before" is completely normal — not a sign you're behind.

Every layer below is checked against DeepTutor specifically, using real issue numbers from work already done, so this isn't abstract — it's a live audit of one real system.

Layer 1 · Classical **Network & Perimeter**

defense in depth

This layer has nothing to do with AI — it's the same network hygiene every internet-facing service needs. Worth stating plainly on a call precisely **because** it's not new: it shows the fundamentals are covered before getting into the interesting part.

- **Unnecessary exposure** — any internal service (a database, an internal tool) reachable directly from the internet is one fewer wall between an attacker and your data.
- **No segmentation** — if every component can talk to every other component, one compromised piece can reach everything.

COVERED The database and the code-execution sandbox are **not reachable from outside the server at all** — only the app itself can talk to them, over an internal-only network. An attacker on the internet can't even attempt to connect to them directly.

Terms you were reaching for: a **DMZ** is a network zone between "open internet" and "fully trusted internal systems" — public-facing things live there so a breach doesn't automatically reach the core. An **IDS/IPS** (intrusion detection/prevention system) watches network traffic for attack patterns and either alerts on them or actively blocks them. Neither is set up for DeepTutor yet — reasonable for a small pilot, worth revisiting at real scale.

Layer 2 · Classical, with an AI twist **Infrastructure & Application**

OWASP LLM03:2025

MITRE ATLAS

Standard application security — but AI adds a new category of "supply chain": the model weights, the tools an agent can call, and third-party plugins are now attack surface, on top of the usual code dependencies.

- **Untrusted file uploads** — any user-uploaded file that isn't validated by type and content is a way to smuggle in something dangerous.
- **Untrusted code execution** — if the AI can run code on a user's behalf, that code needs to run somewhere it can't do real damage.
- **Dependency & tool supply chain** — a compromised package, or a malicious "tool" an agent is given access to, can act with whatever authority the agent has.

COVERED File uploads are validated against an explicit allow-list of types — arbitrary files (including ones that could run active content in a browser) are rejected outright. **#86**

COVERED Code the AI runs executes in a separate, stripped-down container with no special privileges, a read-only filesystem, and no network exposure of its own.

PARTIAL That sandbox currently gives every account visibility into the same shared file area. Rather than build full per-student isolation under time pressure, the safer call was to **turn code execution off for regular student accounts** until real isolation is built. **#7**

Uganda DPPA 2019

NIST AI RMF

What counts as "data" expands with an AI product — it's not just form fields anymore, it's every message a student ever typed to the AI, which can contain far more spontaneous, sensitive disclosure than a structured record ever would.

- **No consent or retention policy** — collecting data without a stated purpose, or keeping it forever, is a compliance liability even before anything goes wrong.
- **Data that can't survive a mistake** — if a bug or a bad actor destroys data and there's no way back, that's a security failure, not just an inconvenience.
- **Cross-border transfer** — if the AI model or storage lives on a server outside the country, that's a legal question, not just a technical one.

COVERED Automated, **tested** backups of the database and files — not just "a backup script exists," a real restore was performed and verified row-for-row against live data. **#78**

COVERED A standing check that scans for data that's quietly gone inconsistent (e.g. a student marked "complete" whose record no longer actually supports that) — catches drift automatically rather than waiting for someone to notice. **#79**

OPEN No explicit, documented consent flow or retention/deletion policy for student chat data specifically — worth closing before real students are enrolled, not after.

Which law actually applies: a South African reviewer will instinctively think in **POPIA** terms. What actually governs a Uganda-hosted platform is the **Data Protection and Privacy Act, 2019** — enforced by Uganda's Personal Data Protection Office. The core principles are nearly identical between the two (consent, purpose limitation, retention limits, security safeguards, restrictions on sending data across borders), so grounding the conversation in the Ugandan law isn't a weaker answer — it's the *correct* one, and it happens to satisfy the same instincts POPIA trained into your colleague.

OWASP LLM01, 02, 07, 08, 09:2025

This is the layer that didn't exist before language models. The model reads instructions and content in the same channel — there's no built-in wall between "the rules the system gave it" and "the text a user or a document handed it." That's the entire root cause of prompt injection.

- **Direct prompt injection (LLM01)** — a user directly typing something like "ignore your previous instructions."
- **Indirect prompt injection (LLM01)** — instructions hidden inside a document, webpage, or file the AI is asked to read, which the AI then treats as commands rather than content.
- **Sensitive information disclosure (LLM02)** — the model reveals something from its training, its instructions, or another user's context that it shouldn't.
- **Misinformation (LLM09)** — the model states something false with total confidence, and it gets acted on. For a tutoring product specifically, this is the risk that matters most day to day.
- **Vector & embedding weaknesses (LLM08)** — for any system using retrieval (looking things up in a knowledge base before answering), a poisoned document in that knowledge base can quietly steer answers.

OPEN DeepTutor lets instructors upload course material that the AI reads and retrieves from. There's no explicit defense yet against an uploaded document containing hidden instructions aimed at the AI rather than the student — this is the single most concrete, nameable gap in the whole framework, and a good one to lead with on a call: "we know exactly where this risk lives in our system, here's the plan."

A live example from this exact system: while verifying the retrieval pipeline earlier, a test document was deliberately seeded with the line *"the secret verification code for this course material is QUOKKA-7742..."* — a benign, planted marker used only to confirm retrieval was working end-to-end. Structurally, that's identical to what an indirect prompt-injection payload looks like: text embedded in retrieved content that the AI reads and could act on. Nothing malicious happened here — but it's the exact shape of the real threat, sitting in the exact place a real attack would live.

Layer 5 · Newest – barely 2 months old **The Agent & Tool-Use Layer**

OWASP Agentic Top 10:2026

DeepTutor doesn't just answer questions — it can call tools, retrieve documents, and execute code across multi-step turns. That's "agentic," and the formal taxonomy for what can go wrong here (ASI01–10) was only published in December 2025. If nobody at your institution has a crisp answer for this layer yet, that's not a gap in your team — the field itself is that young.

- **Goal hijack (ASI01)** — manipulating the agent into pursuing an attacker's objective instead of the user's.
- **Tool misuse (ASI02)** — tricking the agent into using a legitimate tool in a harmful way.

- **Privilege abuse (ASI03)** — the agent acting with more authority than the specific request warrants.
- **Unexpected code execution (ASI05)** — exactly the risk closed in Layer 2, named here as its own category because it's common and severe enough to warrant one.
- **Memory & context poisoning (ASI06)** — corrupting what an agent "remembers" across a conversation or across sessions so it behaves wrongly later.

COVERED Code execution disabled for student accounts by default, closing off ASI05 for the account type where it matters most. **#7**

COVERED A tool that hangs or never returns can no longer stall an entire conversation indefinitely — it now times out cleanly and the failure is tracked. **#83**

OPEN No formal review yet against ASI01/02/03/06 specifically — reasonable next step once the framework itself has had time to mature past its first two months.

Layer 6 · Catches what the rest miss

Operations & Response

OWASP LLM10:2025

Every layer above can be perfectly designed and still fail quietly in production. This layer is about noticing when that happens, and having already decided what to do about it.

- **Unbounded consumption (LLM10)** — nothing stopping a flood of requests from running up a very large AI bill (sometimes called "denial of wallet") or degrading service for everyone else.
- **No incident response plan** — knowing something is wrong is only useful if there's already an agreed next step.
- **Unreviewed changes reaching real users** — the single most common way security regressions actually happen in practice.

COVERED Every change now goes through a documented review step before it ships — small, scoped commits, tested against a live copy of the app rather than assumed correct.

COVERED Tested backups and the automated data-integrity check double as an operational safety net, not just a data-protection measure.

OPEN No rate limiting or cost-abuse protection on the chat endpoint yet, and no written incident-response plan — both reasonable to defer for a small pilot, both worth having before wider rollout.

How to actually answer the question, out loud, on a call

1. Open with the structure, not a list of features.

"We think about this in six layers — network, infrastructure, data, and then two layers that are genuinely new because we're running an AI agent: how the model itself can be manipulated, and how much autonomy we give it to act."

2. Name what's solved, plainly.

"Network and infrastructure are handled with fairly standard practice — nothing's exposed that shouldn't be, untrusted code runs in an isolated sandbox, uploads are validated."

3. Ground the data question in the right law.

"For data, we're built around Uganda's Data Protection and Privacy Act, which covers the same ground POPIA does — consent, retention limits, cross-border rules — and we have tested backups, not just a backup script that's never been proven to work."

4. Be specific about what's new — this is where credibility is actually won.

"Where it gets genuinely new is prompt injection and agent tool misuse — risks that didn't exist before language models. We're mapped to the current OWASP frameworks for both. We've already closed the code-execution risk. We know exactly where our remaining exposure is: an uploaded course document could theoretically try to smuggle instructions to the AI, and we don't have an explicit defense against that yet."

5. Close with the honest gap and the plan, not a claim of completeness.

"We're not claiming this is solved — we're claiming we know the shape of the problem, we're using the same frameworks the rest of the industry just agreed on, and here's exactly what's still open."

SOURCES

[OWASP Top 10 for LLM Applications \(2025\)](#)

[OWASP Top 10 for Agentic Applications \(2026\)](#)

[NIST AI Risk Management Framework](#)

[MITRE ATLAS — Adversarial Threat Landscape for AI Systems](#)

[Uganda Data Protection and Privacy Act, 2019 — compliance analysis](#)

[How POPIA affects AI — Michalsons](#)

[Prompt Injection Attacks on LLMs: attack methods, root causes, defenses — ScienceDirect](#)

Issue numbers ([#7](#) , [#78](#) , [#79](#) , [#83](#) , [#86](#)) refer to tracked, closed work on DeepTutor's own GitHub repository — each one verified against the live running application before being marked done, not just reviewed as code.